

INTRANET 3 - HTTPS via NGINX et log

Auteur

Nom : [DEMAISON Camille]

Dans cette troisième partie on a mis en place la connection sécurisée entre le serveur WEB et la client via HTTPS et une gestion des logs sur le serveur WEB.

Pour mettre en place ceci on a du utiliser Gunicorn qui permet de gérer plusieurs requêtes en même temps, cela ajoute de la fluidité si l'on a plusieurs utilisateurs qui veulent se connecter simultanément. Pour gérer l'HTTPS on a utilisé Nginx qui permet d'utiliser le certificat de l'autorité de certification.

Structure du projet :

Actuellement l'application est situé dans différents dossiers avec cette structure :

```
/opt/flask_app/
├─ start_flask.sh          # Script de lancement automatique
├─ wsgi.py                 # Gère proprement l'instance gunicorn
├─ app.py                 # Déjà présent
├─ requirements.txt       # Déjà présent
├─ templates/
│   └─ login.html         # Déjà présent
│   └─ home.html          # Déjà présent
│   └─ publish.html       # Déjà présent
├─ static/
│   └─ favicon.ico        # Déjà présent
└─ instance/              # Déjà présent
    └─ news.db            # Déjà présent

/tmp/flask_sessions/      # Pour gérer les connexions
└─ **session**           # Chaque fichiers équivaut à une session

/etc/nginx/sites-available/ # Pour configurer Nginx
└─ flaskapp              # Configuration de notre site WEB en
HTTP/HTTPS
```

Technologies utilisées :

Voici le tableau mis à jour avec les outils/bibliothèques utilisés actuellement :

Technologie	Usage
Flask	Framework web principal
Flask-Login	Gestion des sessions utilisateurs
Flask-WTF	Gestion sécurisée des formulaires avec CSRF intégré
Flask-SQLAlchemy	ORM pour interagir avec la base de données
flask_ldap3_login	Authentification via LDAP/LDAPS
ldap3	Connexion et opérations LDAP avec support SSL/TLS
WTForms	Définition des champs et validation des formulaires Pythonic
Bootstrap 5	Interface utilisateur moderne et réactive
Bleach	Filtrage des balises d'un texte
Flask-Session	Gestion des utilisateurs par des multi-worker
Gunicorn	Traitement des requêtes par des multi-worker
Nginx	Gestion de la connexion chiffrée avec certificat de la PKI

Bibliothèques installées

On a actuellement besoin d'installer ces bibliothèques pour faire fonctionner cette partie du projet :

```
Flask
Flask-Login
Flask-WTF
Flask-SQLAlchemy
flask-ldap3-login
ldap3
wtforms
bleach
Flask-Session
```

Nouvelle bibliothèque

Installation des outils

On a du installer différents outils pour utiliser l'HTTPS lors de la connexion du client, tels que :

Gunicorn

Actuellement on lance le serveur de développement intégré de Flask qui utilise Werkzeug . Il est très pratique pour développer, mais **il ne doit pas être utilisé en production**, car :

- Il ne gère pas plusieurs connexions en parallèle et pas très efficace ce qui le rend plus lent
- Il ne propose pas de configuration/sécurisation avancée
- Il ne gère pas HTTPS

On a donc du utiliser Gunicorn pour pouvoir faire plusieurs taches en parallèle. Gunicorn utilise un système de worker pour gérer cela et est plus efficace dans le traitement de connexions multiples.

Installation

Pour l'installer il suffit d'entrer ceci :

```
apt install gunicorn -y
```

NGINX

Cette application servira pour configurer certaines parties telles que les redirections, traitement des connections sécurisées et chiffrées (SSL) avec les certificats de l'autorité de certification via HTTPS , etc.

Installation

Pour l'installer il suffit d'entrer ceci :

```
apt install nginx -y
```

wsgi.py — Gestion par Gunicorn

Pour gérer plus efficacement les connexion multiples on utilise `Gunicorn` qui utilise plusieurs worker.

Fichier de démarrage

C'est un **petit fichier d'entrée** que `Gunicorn` utilisera pour charger l'app, situé à la racine du projet avec `app.py` :

```
from app import app, db

with app.app_context():
    db.create_all()

if __name__ == "__main__":
    app.run()
```

On n'a **pas besoin de modifier** `app.py` pour cette partie, vu qu'on a cette condition :

```
if __name__ == '__main__':
```

→ En effet l'application ne sera plus lancé via ce script python donc le main de `app.py` ne sera pas exécuté. On le garde au cas où on a besoin de debug.

Tester avec gunicorn

On se place dans le bon dossier :

```
cd /opt/flask_app
```

Puis on lance :

```
gunicorn -w 4 -b 0.0.0.0:5000 wsgi:app
```

- `-w 4` = 4 workers (processus en parallèles) à ajuster si besoin
- `-b 0.0.0.0:5000` = Ecoute sur tout le réseau local avec le port 5000
- `wsgi:app` = L'application Flask est importé depuis `wsgi.py`

Pour arrêter l'application il suffit d'entrer cette commande :

```
pkill gunicorn
```

Lancer en arrière-plan (car pas de systemd)

```
cd /opt/flask_app
nohup gunicorn -w 4 -b 127.0.0.1:5000 wsgi:app > /var/log/gunicorn.log
2>&1 &
```

- `nohup` => **Empêche l'arrêt** si l'on ferme le terminal (No Hang Up)
- `-w 4 -b 127.0.0.1:5000 wsgi:app` => Ecoute sur le **localhost** avec le **port 5000** et charge l'app Flask déclaré dans `wsgi.py`
- `> /var/log/gunicorn.log 2>&1` => **Redirige les logs ainsi que les erreurs** vers le fichier précisé ici
- `&` => Lance en **arrière-plan**

On peut vérifier que `Gunicorn` tourne avec :

```
ps aux | grep gunicorn
```

On voit que les services de `Gunicorn` tournent bien, si ce n'est pas le cas il faut essayer de voir où ça a planté.

app.py — Gestion des sessions multi-worker

Actuellement nous enregistrons les utilisateurs connectés dans un dictionnaire. Les informations sont donc stockés dans la RAM du programme mais `Gunicorn` utilise plusieurs worker et chaque worker charge le programme de manière indépendante, le dictionnaire n'est donc pas partagé. Donc si l'on se connecte sur le **worker 1** et que lors du chargement de la page suivante on utilise le **worker 2** alors on devra se reconnecter.

Pour éviter cela on a utilisé une nouvelle bibliothèque `Session` ainsi que `timedelta` pour la gestion du temps de connexion.

Pour mettre cela en place on a du importer ces librairies :

```
from flask import Flask, render_template, redirect, flash, url_for,
request, session
from flask_session import Session
from datetime import timedelta
```

On a du ajouter quelques paramètres de configuration de l'application après `app = Flask(__name__)` :

```
app = Flask(__name__)

...

# Gestion des utilisateurs
app.config['SESSION_TYPE'] = 'filesystem'
app.config['SESSION_FILE_DIR'] = '/tmp/flask_sessions'
app.config['SESSION_PERMANENT'] = True
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(hours=4) # minutes=1
app.config['SESSION_REFRESH_EACH_REQUEST'] = False

# Gestion des cookies securise
app.config['SESSION_COOKIE_SECURE'] = True
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'

Session(app)
```

- **SESSION_TYPE** = 'filesystem' => Indique que Flask doit stocker les données de session sur le disque (et non en mémoire ou en base de données).
- **SESSION_FILE_DIR** = '/tmp/flask_sessions' => Spécifie le dossier où seront stockés physiquement les fichiers de session.
- **SESSION_PERMANENT** = True => Rend la session "permanente", c'est-à-dire qu'elle a une durée de vie définie par `PERMANENT_SESSION_LIFETIME` . Sinon, elle expire quand l'utilisateur ferme le navigateur.
- **PERMANENT_SESSION_LIFETIME** = `timedelta(hours=4)` => Définit la durée de vie d'une session permanente : ici, **4 heures**.

- **SESSION_REFRESH_EACH_REQUEST** = False => Dit à Flask de **ne pas** prolonger la durée de vie de la session à chaque requête.
- **SESSION_COOKIE_SECURE** = True => Fait en sorte que les cookies de session ne soient envoyés **que via HTTPS**.
- **SESSION_COOKIE_HTTPONLY** = True => Rend les cookies de session **inaccessibles** en JavaScript (`document.cookie` ne pourra pas y accéder).

C'est une protection en plus contre certaines attaques XSS.

- **SESSION_COOKIE_SAMESITE** = 'Lax' => Contrôle l'envoi des cookies dans des utilisations cross-site.

'Lax' : protège contre certaines attaques CSRF tout en permettant l'envoi des cookies.

On supprime donc le dictionnaire car il ne sera plus utile :

```
# users = {}
```

On a donc du modifier `save_user()` pour pouvoir l'utiliser avec la nouvelle bibliothèque :

```
@ldap_manager.save_user
def save_user(dn, username, data, memberships):
    # Groupe requis pour avoir acces a l'application
    required_group = 'CN=GRP-INTRANET,OU=Groupes,OU=Service
ITway,DC=itway,DC=local'

    # Verification de l'appartenance au groupe requis
    if 'memberOf' in data and required_group in data['memberOf']:
        session['user_dn'] = dn
        session['user_username'] = username
        session['user_data'] = data
        return User(dn, username, data)

    # Refus si l'utilisateur ne fait pas partie du groupe autorise
    return None
```

Ici on utilise la bibliothèque `Session` pour sauvegarder un utilisateur spécifique. Pas besoin de réexpliquer, on suit la même logique que précédemment.

On a fait de même pour la gestion des chargement des utilisateurs :

```
@login_manager.user_loader
def load_user(id):
    dn = session.get('user_dn')
    username = session.get('user_username')
    data = session.get('user_data')

    if dn == id and username:
        return User(dn, username, data)
    return None
```

Modification la route :

```
# Route pour la page de connexion
@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('home')) # Redirige vers l'accueil si
l'utilisateur est deja connecte

    form = LDAPLoginForm()

    if form.is_submitted():
        if contains_ldap_special_chars(form.username.data.strip()):
            return render_template('login.html', form=form) # rejet
silencieux

        if form.validate():
            user = form.user
            if user:
                login_user(user)
                session.permanent = True # Ajout ici pour activer une
session de 4h
                next_page = request.args.get('next') # Recupere l'URL de
redirection
                return redirect(next_page or url_for('home'))
            else:
                flash("Vous n'avez pas les droits pour accéder à cette
application.")
            else:
                flash("Nom d'utilisateur ou mot de passe incorrect.")

    return render_template('login.html', form=form)
```

- On a du ajouter `session.permanent = True` :

On l'ajoute ici pour activer une session de 4h, c'est l'initialisation du timer.

⚠ Warning

Si l'on passe par `Gunicorn` ou par le script python alors on ne sera plus en HTTPS, il faudra donc commenter les trois attributs gérant les cookies car ils forcent l'utilisation de l'HTTPS

```
# Gestion des cookies
app.config['SESSION_COOKIE_SECURE'] = True
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
```

ℹ Info

On a du ajouter une gestion des erreurs 404 spécifique car sinon on obtenait une boucle, en effet il se rappelait récursivement de manière infinie. (gestion des erreurs générale par le refresh de la page avec un message)

```
# Gestion des erreurs 404 (page non trouvee)
@app.errorhandler(404)
def page_not_found(e):
    return render_template('404.html'), 404
```

Une page `404.html` a donc été ajouté dans le dossier `templates`.

flaskapp — Configuration de NGINX avec HTTPS

Vérification des certificats SSL

On vérifie que nos certificats signés par notre autorité interne sont bien ici :

```
/etc/ssl/certs/wildcard.cert.pem;
/etc/ssl/private/wildcard.key.pem;
```

Configuration de NGINX

On a donc du créer la config dans ce fichier de configuration :

```
/etc/nginx/sites-available/flaskapp
```

Le fichier contient cette configuration :

```
# Redirige TOUT le HTTP vers HTTPS sur le domaine principal
server {
    listen 80;
    server_name srv-intranet srv-intranet.itway.local intranet
intranet.itway.local;

    return 301 https://intranet.itway.local$request_uri;
}

# Redirige les mauvais domaines HTTPS vers le bon
server {
    listen 443 ssl;
    server_name srv-intranet srv-intranet.itway.local intranet;

    ssl_certificate /etc/ssl/certs/wildcard.cert.pem;
    ssl_certificate_key /etc/ssl/private/wildcard.key.pem;
    ssl_trusted_certificate /usr/local/share/ca-certificates/myCA.crt;

    ssl_protocols TLSv1.2 TLSv1.3;

    return 301 https://intranet.itway.local$request_uri;
}

# Le vrai serveur qui sert l'app
server {
    listen 443 ssl;
    server_name intranet.itway.local;

    ssl_certificate /etc/ssl/certs/wildcard.cert.pem;
    ssl_certificate_key /etc/ssl/private/wildcard.key.pem;
    ssl_trusted_certificate /usr/local/share/ca-certificates/myCA.crt;

    ssl_protocols TLSv1.2 TLSv1.3;

    error_log /var/log/nginx/error.log;
    access_log /var/log/nginx/access.log;
```

```

location / {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

- Dans les deux premiers blocs on redirige les différents URL en HTTP et HTTPS vers :

`https://intranet.itway.local$request_uri` => Cela permet que tout les utilisateurs aient le même URL final en HTTPS, cela simplifie la gestion des cookies et uniformise le tout

- Dans le troisième bloc on gère la connection au bon URL :
 - On écoute sur le port 443, port de l'HTTPS
 - On indique les chemins des certificats
 - On enregistre les différents logs dans les fichiers correspondants
 - On dit a Nginx de reverse-proxy vers l'application flask (donc Gunicorn)

Ensuite on active et lance la configuration précédente :

```

ln -s /etc/nginx/sites-available/flaskapp /etc/nginx/sites-enabled/
nginx
nginx -t && nginx -s reload # Car pas de systemd

```

La première ligne permet d'activer la configuration.

La deuxième ligne lance Nginx au cas où il ne serait pas déjà lancé.

La dernière ligne permet de vérifier si la configuration n'a pas d'erreur et de recharger Nginx.

Info

Attention a bien supprimer la conf par défaut : ça permet d'éviter d'être parasité.
En effet la config par défaut écoute sur le port 80 et donc peut avoir des comportements inattendus.

Elle est à ces emplacements :

```

/etc/nginx/sites-available/default
/etc/nginx/sites-enabled/default

```

start_flask.sh — Script de démarrage

Pour éviter de taper manuellement `nohup gunicorn...`, on a créé un script :

```
/opt/flask_app/start_flask.sh
```

On a d'abord fait un script minimal qui contenait :

```
#!/bin/bash
cd /opt/flask_app
nohup gunicorn -w 4 -b 127.0.0.1:5000 wsgi:app > /var/log/gunicorn.log
2>&1 &
nginx
nginx -t && nginx -s reload
```

Pour pouvoir suivre plus clairement et afficher les étapes on a fait un petit script amélioré :

```
#!/bin/bash

# Chemin vers notre application Flask
APP_DIR="/opt/flask_app"
GUNICORN_LOG="/var/log/gunicorn.log"
NGINX_SITE_AVAILABLE="/etc/nginx/sites-available/flaskapp"
NGINX_SITE_ENABLED="/etc/nginx/sites-enabled/flaskapp"

echo "=== Demarrage du serveur Flask avec Gunicorn ==="

# Aller dans le dossier de l'app
cd "$APP_DIR" || { echo "Erreur: Impossible d'accéder à $APP_DIR"; exit 1; }

# Verifie si Gunicorn tourne deja
if pgrep gunicorn > /dev/null; then
    echo "Gunicorn est déjà en cours d'exécution. Arrêt en cours..."
    pkill gunicorn
    sleep 2
fi

# Lancer Gunicorn en arriere-plan
echo "Lancement de Gunicorn..."
nohup gunicorn -w 4 -b 127.0.0.1:5000 wsgi:app > "$GUNICORN_LOG" 2>&1 &

# Configurer Nginx si necessaire
if [ ! -f "$NGINX_SITE_ENABLED" ]; then
    echo "Lien symbolique Nginx absent, creation..."
    ln -s "$NGINX_SITE_AVAILABLE" "$NGINX_SITE_ENABLED"
else
```

```

    echo "Lien symbolique Nginx deja present."
fi

# Verifier la config Nginx
echo "Verification de la configuration Nginx..."
if nginx -t; then
    # Demarrer Nginx si pas demarre
    if ! pgrep nginx > /dev/null; then
        echo "Nginx non demarre, demarrage..."
        nginx
    fi
    echo "Reload de Nginx..."
    nginx -s reload
else
    echo "Erreur dans la configuration Nginx, abandon."
    exit 1
fi

echo "=== Demarrage termine avec succes ==="

```

On le rend exécutable :

```

chmod +x /opt/flask_app/start_flask.sh

```

Puis on lance l'app avec :

```

cd /opt/flask_app
./start_flask.sh

```

Maintenant qu'on sait qu'il fonctionne correctement il ne reste plus qu'a l'intégrer au démarrage de la machine.

Pour automatiser le lancement de certains services dans un conteneur GNS3 on utilise le fichier `/gns3/run-cmd.sh`, ce fichier est lancé automatiquement au démarrage d'un conteneur. On ajoute donc notre script pour qu'il soit exécuté au démarrage :

```

#!/bin/sh
# run docker startup, first arg is new PATH, remainder is command

PATH="$1"
shift

```

```
echo "Starting services..."
# Démarrer BIND en arrière plan :
nohup /usr/sbin/named -4 -u bind -g > /var/log/named.log 2>&1 &
# Démarrer le serveur SSH :
service ssh start &
# Démarrer Wazuh
service wazuh-agent start &

cd /opt/flask_app
./start_flask.sh

exec "$@"
```

Info

Dans le script on se place dans le dossier `flask_app` car lors du lancement de `gunicorn` il va créer un fichier, je voulais qu'il se crée dans notre dossier pour tout centraliser.

Tester le site

Soit on relance le docker soit sinon on exécute ces commandes dans le terminal

```
cd /opt/flask_app
./start_flask.sh
```

Ensuite on se connecte depuis un poste client qui a accès au certificat de notre autorité :

```
https://intranet/

# ou
https://intranet.itway.local/
```

Tests et vérifications

Pour vérifier que tout est bien on va devoir vérifier certains points :

- La machine est opérationnelle automatiquement après le démarrage
- La page d'erreur 404 personnalisée est utilisée correctement
- Contrôle du chiffrement de la communication via une analyse wireshark

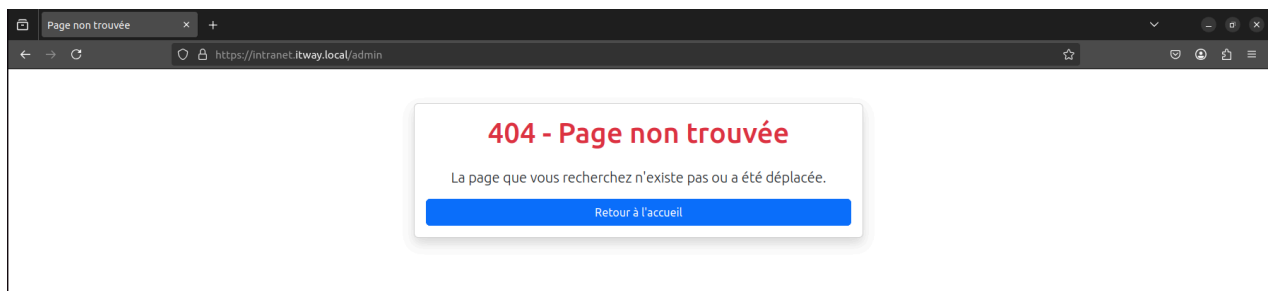
Différents tests ont été procédé :

- Lancement automatique du serveur :
Pour le tester on a juste à stopper la machine et de la relancer.

En se connectant en SSH on constate bien que le script personnalisé est bien lancé que le site est de nouveau joignable. Le temp de lancement étant rapide c'est un succès.

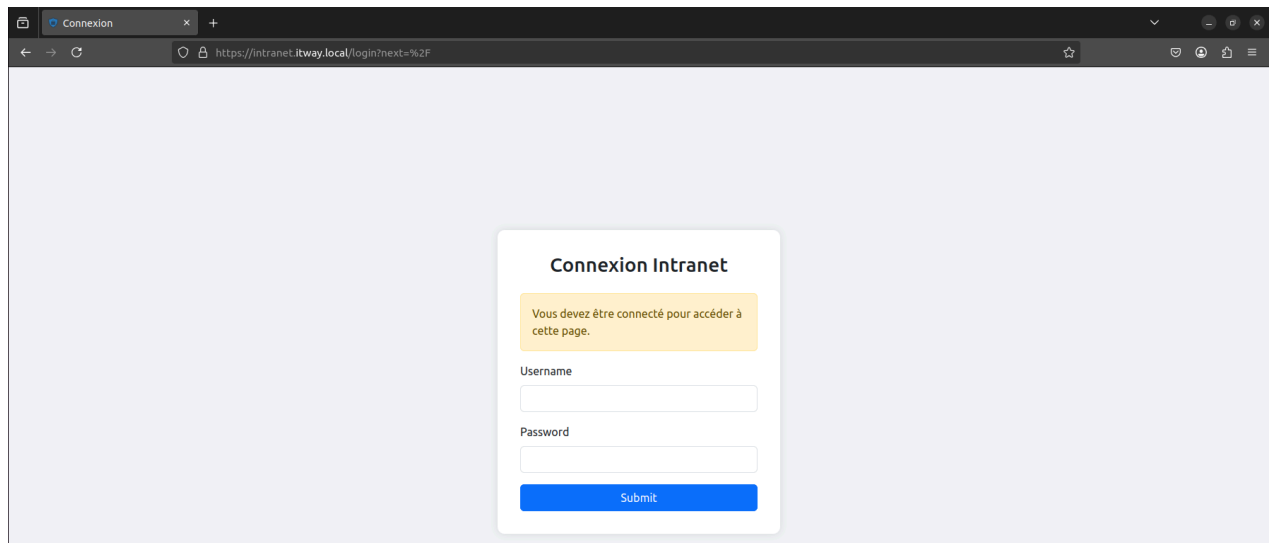
- Test d'affichage de la page d'erreur 404 et la redirection à la page principale :
On se connecte sur la page admin : (qui n'existe pas)
 - <http://srv-intranet:5000/admin>

On obtient ce résultat :



On voit que la page d'erreur 404 qui correspond à l'erreur lorsqu'une page n'existe pas apparaît ce qui est normal.

Lorsqu'on clique sur le bouton de redirection vers la page d'accueil on obtient ceci:



On arrive alors sur la page de connexion car l'utilisateur n'était pas connecté. Si l'on se connecte on arrive bien sur la page d'accueil.

- Vérification du chiffrement de la communication client serveur :

Pour cela on a utilisé Wireshark et l'on a fait une capture au moment d'une requête d'une page quelconque :

Appliquer un filtre d'affichage ... Ctrl+L						
No.	Time	Source	Destination	Protocol	Length	Info
10.000000	192.168.10.14	192.168.20.4	TCP	66	60782 → 443 [ACK] Seq=1 Ack=1 Win=573 Len=0 TSval=2095861863 TSecr=4078315116	
20.000241	192.168.20.4	192.168.10.14	TCP	66	TCP ACKed unseen segment 443 → 60782 [ACK] Seq=1 Ack=2 Win=501 Len=0 TSval=4078325350 TSecr=2095800240	
30.511079	192.168.10.14	192.168.20.4	TCP	66	60782 → 443 [ACK] Seq=1 Ack=1 Win=666 Len=0 TSval=2095862375 TSecr=4078315629	
40.951213	192.168.20.4	192.168.10.14	TCP	66	TCP ACKed unseen segment 443 → 60782 [ACK] Seq=1 Ack=2 Win=501 Len=0 TSval=4078325800 TSecr=2095800908	
52.653970	192.168.20.4	192.168.60.2	TCP	368	60535 → 1514 [PSH, ACK] Seq=1 Ack=1 Win=502 Len=302 TSval=1048493638 TSecr=3002571639	
62.657896	192.168.60.2	192.168.20.4	TCP	155	1514 → 60535 [PSH, ACK] Seq=1 Ack=303 Win=1279 Len=89 TSval=3002581651 TSecr=1048493638	
72.657270	192.168.20.4	192.168.60.2	TCP	66	60535 → 1514 [ACK] Seq=303 Ack=90 Win=502 Len=0 TSval=1048493641 TSecr=3002581651	
83.375871	192.168.10.14	192.168.20.4	TLSv1.2	576	TCP Previous segment not captured in Application Data	
95.376076	192.168.20.4	192.168.10.14	TCP	66	443 → 60782 [ACK] Seq=1 Ack=511 Win=501 Len=0 TSval=4078330732 TSecr=2095867238	
105.405054	192.168.20.4	192.168.10.14	TLSv1.2	1224	Application Data	
115.410033	192.168.10.14	192.168.20.4	TCP	66	60782 → 443 [ACK] Seq=511 Ack=1159 Win=666 Len=0 TSval=2095867273 TSecr=4078330761	
126.626539	192.168.20.4	192.168.60.2	TCP	304	60535 → 1514 [PSH, ACK] Seq=303 Ack=90 Win=502 Len=238 TSval=1048497611 TSecr=3002581651	
136.670029	192.168.60.2	192.168.20.4	TCP	66	1514 → 60535 [ACK] Seq=90 Ack=541 Win=1279 Len=0 TSval=3002585664 TSecr=1048497611	
1410.239741	192.168.10.14	192.168.20.4	TCP	66	TCP Dup ACK 1#1 60782 → 443 [ACK] Seq=1 Ack=1 Win=573 Len=0 TSval=2095872163 TSecr=4078325356	
1510.233097	192.168.20.4	192.168.10.14	TCP	66	TCP Dup ACK 2#1 443 → 60782 [ACK] Seq=1 Ack=2 Win=501 Len=0 TSval=4078335509 TSecr=2095800240	

Ici le handshake (échange de clés) n'a pas été entièrement capturé ce qui affiche une erreur mais on constate quand même que la communication entre le client (ici 192.168.10.14) le serveur INTRANET (192.168.20.4) est **chiffré avec TLSv1.2**.

L'autre communication relevée est la communication du serveur INTRANET avec l'AD pour l'authentification via LDAPs.

Si on analyse le flux on obtiens bien un échange chiffré, même sur les paquets n'indiquant que TCP :

[illegible]

Il y a bien l'échange de clef puis les données sont chiffrées.